

OCD: Object Change Distillation for One-Shot Multi-Edit Image Editing

Anonymous submission

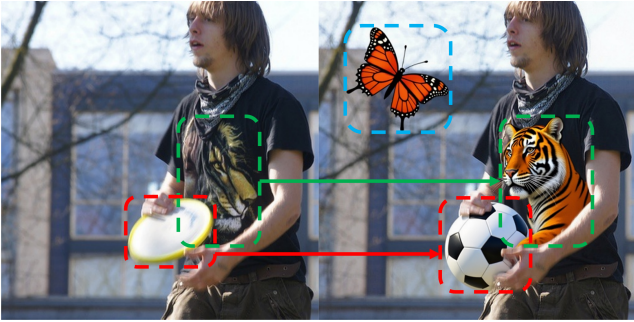


Figure 1: One-shot multi-edit image editing. One instruction adds a butterfly, replaces the frisbee with a soccer ball, and changes the shirt pattern. The output must realize all three requests in one coherent image. Standard final-image supervision shows the joint result but does not explicitly identify which visible change corresponds to each request.

Abstract

Real image-editing instructions often combine several changes that must appear together, yet final-image supervision does not identify which visible change corresponds to each request. We introduce OCD (*Object Change Distillation*), which uses a structured editor only during training to recover this missing supervision. The teacher produces a complete edit and one leave-one-request-out variant per request while keeping the remaining requests active; their difference exposes that request’s change in the joint context. A shared student learns these signals alongside the complete result, then executes the full instruction through one standard image-and-text call without masks, planning, or sequential editing. Across SDXL and FLUX backbones, OCD improves atomic-request completion on ComplexBench-Edit and instruction-to-object binding on OIR/LoMOE. On a single A800, OCD-SDXL runs in 3.67 seconds per image, a $7.2\times$ speedup over LoMOE and $12.7\times$ over LayerEdit. Among the evaluated ablations, the complete training configuration gives the strongest joint semantic alignment and binding.

1 Introduction

Instruction-guided image editing is appealing because of its direct interface: a user provides a source image and describes the desired result in natural language (Brooks, Holynski,

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

and Efros 2023; Zhang et al. 2023). In practice, however, one instruction often bundles several changes. A user may ask to replace a cat with a dog, add a collar to the new dog, recolor the collar, and remove an unrelated object. The desired output is not a sequence of intermediate images, but one coherent image that satisfies the complete instruction. We study this setting as *one-shot multi-edit image editing*: the editor receives the source image and complete instruction once, and must realize every requested change in the returned image. The requests may concern different objects, modify the same object several times, or depend on an earlier change (Figure 1).

The difficulty is not simply that the instruction is longer. Standard editors are trained with a final edited image that shows where the scene should end, but does not explicitly show how each part of the instruction produced that result. For a single edit, this link may be easy to infer. For a multi-edit instruction, a final image containing a dog with a red collar does not indicate which phrase replaced the cat, which introduced the collar, or which changed its color. The instruction contains N atomic requests, but training provides only one final target. As a result, the target does not clearly show which phrase should change which object. This mismatch produces three recurring failures at test time: a request is omitted, a change is assigned to the wrong object, or one edit disrupts another.

Existing approaches primarily address these failures by changing how the edit is executed. Sequential methods apply atomic requests one after another, which can accumulate reconstruction drift and earlier editing errors (Kim, Lee, and Park 2025). Structured methods separate requests using masks, layers, localized streams, or region-specific prompts (Chakrabarty et al. 2024; Huang et al. 2024a; Zhu et al. 2025; Fu et al. 2026), while planning-based systems infer such structure at test time (Schouten et al. 2025; Qu et al. 2025). These approaches provide useful control, but the separated requests and spatial inputs must be created and used for every new image. A different option is to train on isolated single-edit examples. Such examples make individual changes easier to observe, but remove the joint context in which the changes must coexist. This raises a different question: can individual changes be exposed during training while the editor still receives the complete instruction only once at test time?

We introduce OCD, *Object Change Distillation*, based on using structure to teach rather than to edit. A structured teacher produces a matched no-edit output (all requests disabled), a complete output, and one leave-one-request-out output for each request. The complete output minus the no-edit output defines the complete change; the complete output minus a version omitting only request i asks what disappears when that request is removed. Because all other requests remain active in the latter pair, the target retains their joint context without assuming that independently generated edits add up to the complete result.

The per-request targets and the complete target play different roles. In a per-request training example, the shared student connects one atomic phrase to its visible change. In a complete example, the same student receives the full instruction and learns the final joint edit. The first type shows which change belongs to each phrase; the second teaches how the changes coexist and handles dependencies that one phrase cannot describe by itself. Both types train the same model, so no separate module is needed at test time. The teacher, masks, separated prompts, and leave-one-out runs are removed. The student receives only the source image and complete instruction, once.

We evaluate this design on both SDXL and FLUX.1 Kon-text backbones (Podell et al. 2024; Labs et al. 2025). On ComplexBench-Edit (Wang et al. 2025), OCD improves the completion of separately scored atomic requirements; on the OIR/LoMOE protocol (Yang et al. 2024; Chakrabarty et al. 2024), it improves instruction-to-object binding relative to the corresponding pretrained backbones. The training ablation separates the amount of supervision from the way its targets are built. Complete-output and composed isolated targets improve global semantic alignment but reduce binding, while isolated single-edit targets provide a smaller binding gain. Among the tested settings, the full model with paired leave-one-out supervision gives the strongest joint improvement in semantic alignment and binding. These results suggest that request-level targets built in the full editing context work best together with complete-instruction training.

Our contributions are threefold:

- We identify a supervision mismatch in one-shot multi-edit image editing: a complete instruction contains several atomic requests, but its final-image target does not show which visible change belongs to each request.
- We introduce OCD, which builds a complete change target from matched complete/no-edit teacher outputs and one target per request from matched complete/leave-one-request-out outputs, then trains one shared editor with both.
- We apply OCD to SDXL and FLUX backbones and evaluate both atomic-request completion and instruction-to-object binding, showing consistent improvements without masks, planning, or sequential editing at test time; on a single A800, OCD-SDXL is $7.2\times$ faster than LoMOE and $12.7\times$ faster than LayerEdit.

2 Related Work

Instruction-guided image editing. Instruction-guided editors learn to transform a source image from a natural-language request using synthetic or curated edit pairs (Brooks, Holynski, and Efros 2023; Zhang et al. 2023; Sheynin et al. 2024; Zhao et al. 2024; Yu et al. 2025). Later work improves task coverage, data quality, architecture, or backbone capacity (Wei et al. 2025; Chen, Sra, and Sen 2025; Labs et al. 2025; Huang et al. 2024b). These methods define the image-plus-instruction interface that we keep at test time. Rather than enlarging the editor or training set, we change how targets for a multi-edit instruction are built while keeping the student’s test-time input and sampling process unchanged.

Executing multiple edits. Existing multi-edit methods separate requests during editing. Regional editors use masks or modify attention to keep edits local (Avrahami, Lischinski, and Fried 2022; Couairon et al. 2023; Guo and Lin 2024). ParallelEdits and IID separate multiple instructions through model branches or attention maps (Huang et al. 2024a; Liu et al. 2025), while MoEdit, LoMOE, and MDE-Edit focus on edit count, object instances, or local optimization (Li et al. 2025b; Chakrabarty et al. 2024; Zhu et al. 2025). LayerEdit splits an image into object layers and then merges them (Fu et al. 2026). Planning-based systems create sub-instructions or spatial controls for complex requests (Schouten et al. 2025; Qu et al. 2025; Yeh et al. 2026), while attention guidance reduces missing objects and incorrect attribute binding (Chefer et al. 2023; Rassin et al. 2023). All of these methods separate requests during the editing process. In contrast, OCD uses a structured editor only to build offline training targets; at test time, the student is a standard editor that receives the source image and complete instruction.

Training-time structured supervision. Specialist distillation and supervision refinement improve instruction-based editors without keeping the teacher at test time. OmniEdit collects training pairs from task-specific specialists (Wei et al. 2025), while SuperEdit corrects instruction-image alignment and RefEdit improves referring-expression grounding (Li et al. 2025a; Pathiraja et al. 2025). More broadly, knowledge distillation and learning with extra information available only during training transfer teacher signals into a simpler test-time model (Hinton, Vinyals, and Dean 2015; Vapnik and Vashist 2009). OCD differs in its training targets: rather than directly supervising token-region alignment (Wang et al. 2024), distilling only a specialist’s final output, or correcting the paired instruction, we compare matched complete/no-edit and complete/leave-one-request-out teacher outputs to obtain complete and per-request visual changes. Table 1 summarizes this distinction.

3 Method

Figure 2 summarizes OCD: the structured teacher supplies per-request change targets, while complete-instruction training teaches the final joint edit.

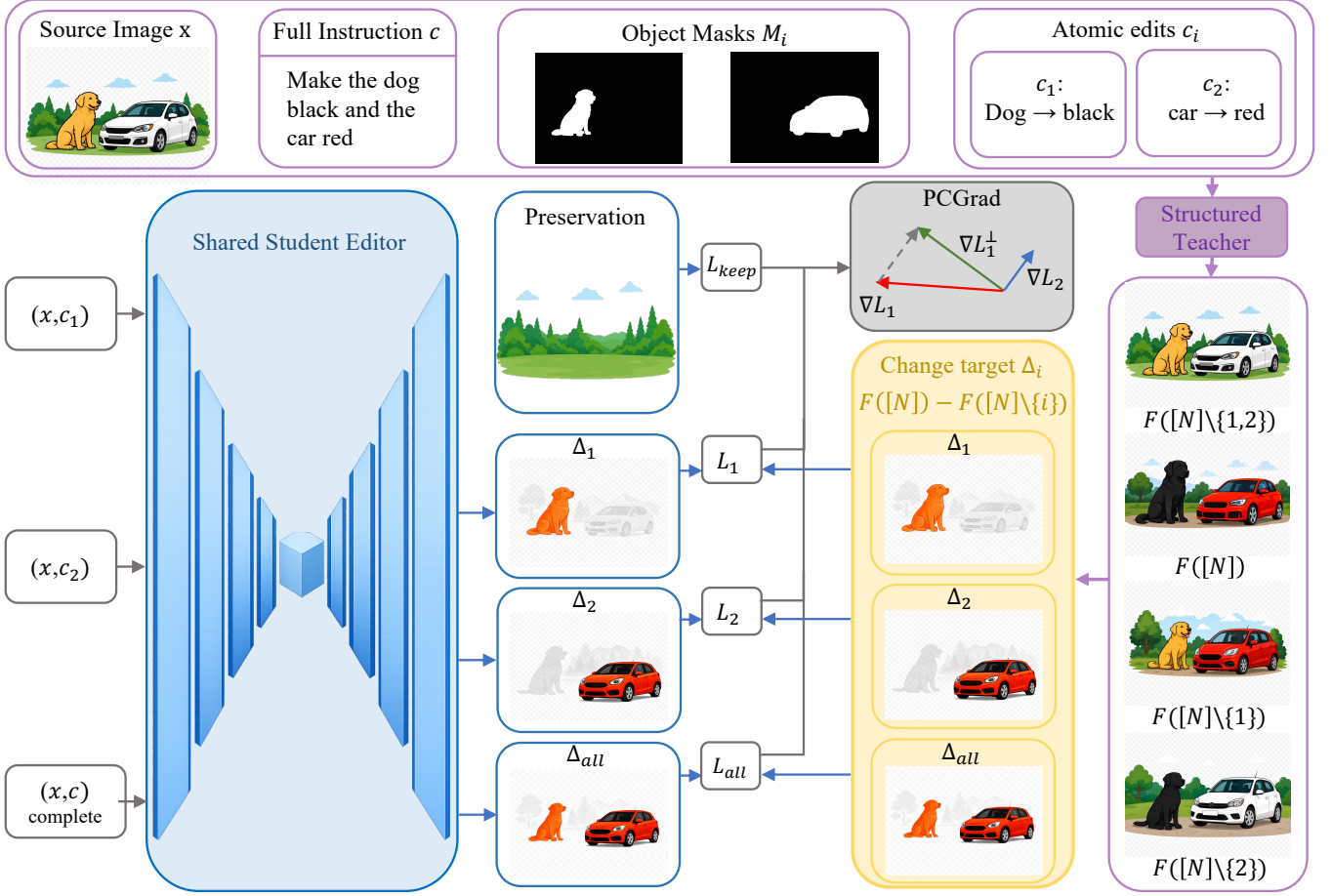


Figure 2: **Object Change Distillation.** Teacher states define $\Delta_{\text{all}} = F([N]) - F(\emptyset)$ and $\Delta_i = F([N]) - F([N] \setminus \{i\})$; for the shown $N = 2$ case, $F([N] \setminus \{1, 2\}) = F(\emptyset)$. Blue tiles estimate the matching orange targets. $\mathcal{L}_1$ and $\mathcal{L}_2$ are two examples of $\mathcal{L}_i$, while $\mathcal{L}_{\text{keep}}$ combines the preservation losses. PCGrad handles conflicting gradients while updating one shared student; at test time, only (x, c) is used.

Family	Training target	Extra input at test	Per-request
Paired instruction editing	final image	no	no
Final-output distillation	teacher final	no	no
Structured execution	varies	yes	varies
OCD (ours)	complete/no-edit complete/leave-one-out	no	yes

Table 1: Comparison of training targets and test-time inputs. OCD builds complete and per-request targets from matched teacher outputs, but does not use this structure at test time.

Training Setup and Structured Teacher

Let x be a source image and let the complete instruction c contain N atomic requests $c_1, \dots, c_N$. During training only, request c_i is associated with an object mask M_i ; masks may overlap when several requests concern the same object. The student receives neither the separated requests nor the masks

at test time. Its only input is (x, c) .

We use LayerEdit (Fu et al. 2026) as the structured teacher; the method only requires an editor that can turn a selected set of requests on or off. Let $[N] = \{1, \dots, N\}$ and let $S \subseteq [N]$ be the active set. The teacher produces

$$y_S^T = \mathcal{T}(x, p_0, \{(c_i, M_i)\}_{i=1}^N, S; \omega), \quad (1)$$

where p_0 describes the source scene and ω fixes the inversion state, scheduler, random seed, resolution, masks, and decoder. Using the same settings for both outputs in each pair reduces unrelated differences. For each example, we precompute the no-edit output $y_{\emptyset}^T$ with all requests disabled, the complete output $y_{[N]}^T$, and one leave-one-request-out output $y_{[N] \setminus \{i\}}^T$ for each request. Thus, the cache contains $N + 2$ teacher outputs per example and grows linearly with N .

All teacher outputs are encoded with the student’s VAE encoder: $F(S) = \mathcal{E}(y_S^T)$. Figure 2 shows the teacher images as thumbnails and labels their encoded states $F(S)$, which are the quantities used to construct the targets.

Complete and Per-Request Change Targets

The complete target and each request target are, respectively,

$$\Delta_{\text{all}} = F([N]) - F(\emptyset), \quad (2)$$

$$\Delta_i = F([N]) - F([N] \setminus \{i\}). \quad (3)$$

The first difference compares the complete output with its matched no-edit output; for Figure 2’s $N = 2$ example, this baseline is $F([N] \setminus \{1, 2\}) = F(\emptyset)$. In the second, all other requests remain active, so Δ_i records what disappears when c_i is removed from that joint context rather than an isolated execution. Interactions can therefore influence Δ_i , but the complete target is not assumed to equal a sum of request targets.

We enlarge mask M_i to M_i^+ so that the target includes nearby boundary changes. A request pair is removed when much of its difference lies outside this enlarged mask, but the example is still used for complete-instruction training. Construction details and cache counts appear in the supplement.

Distilling One Shared Editor

The shared editor is trained with two types of examples. A per-request example (x, c_i) uses Δ_i to connect phrase c_i with its visible effect. Because this input omits the other requests, it cannot handle dependencies by itself. A full example (x, c) uses Δ_{all} and learns the complete edit through the same input used at test time. Both types update the same parameters; no request-specific head is added.

For either a diffusion or flow-matching backbone, let $z_x = \mathcal{E}(x)$. The native prediction at timestep t is converted to a clean latent estimate $\hat{z}_0$, the estimated final-image latent. The student-predicted change relative to the source is

$$\hat{\Delta}_\theta(x, u, t) = \hat{z}_0(x, u, t) - z_x, \quad (4)$$

where $u \in \{c_i, c\}$. Let $\hat{\Delta}_i = \hat{\Delta}_\theta(x, c_i, t)$ and $\hat{\Delta}_{\text{all}} = \hat{\Delta}_\theta(x, c, t)$. These are Figure 2’s blue estimates; hats distinguish them from the orange targets. The per-request loss is

$$\mathcal{L}_i = q_i w_t \frac{\|M_i^+ \odot (\hat{\Delta}_i - \Delta_i)\|_1}{A(M_i^+)}, \quad (5)$$

where $q_i = 1$ only for an accepted pair, w_t is a timestep weight with clipped signal-to-noise ratio (SNR), and $A(M) = \max(\|M\|_1, \tau_{\text{area}}) + \varepsilon$. Figure 2’s $\mathcal{L}_1$ and $\mathcal{L}_2$ are examples of $\mathcal{L}_i$; we write their sum as $\mathcal{L}_{\text{each}} = \sum_i \mathcal{L}_i$. The complete-instruction objective is

$$\mathcal{L}_{\text{all}} = w_t \frac{\|M^+ \odot (\hat{\Delta}_{\text{all}} - \Delta_{\text{all}})\|_1}{A(M^+)}, \quad (6)$$

where M^+ is the union of enlarged masks. This path covers dependencies and joint effects that the per-request input cannot specify. Let $B = 1 - M^+$ and $B_i = 1 - M_i^+$. A preservation term discourages changes outside the edit masks in

both training paths:

$$\mathcal{L}_{\text{keep}} = w_t \frac{\|B \odot \hat{\Delta}_{\text{all}}\|_1}{A(B)} + \sum_i q_i w_t \frac{\|B_i \odot \hat{\Delta}_i\|_1}{A(B_i)}. \quad (7)$$

Figure 2’s single $\mathcal{L}_{\text{keep}}$ box denotes this combined loss. The total objective is

$$\mathcal{L} = \lambda_e \mathcal{L}_{\text{each}} + \lambda_a \mathcal{L}_{\text{all}} + \lambda_k \mathcal{L}_{\text{keep}}. \quad (8)$$

These losses share parameters and their gradients may conflict, so we apply PCGrad (Yu et al. 2020) to $\{\lambda_e \mathcal{L}_i : q_i = 1\} \cup \{\lambda_a \mathcal{L}_{\text{all}}, \lambda_k \mathcal{L}_{\text{keep}}\}$. Figure 2 illustrates one conflicting pair, $\nabla \mathcal{L}_1$ and $\nabla \mathcal{L}_2$. PCGrad is used only during training.

One-Shot Editing at Test Time

At test time every training-only component is removed. Given the source image and the complete instruction, the student runs its backbone’s standard sampling procedure once:

$$\hat{y} = S_\theta(x, c), \quad (9)$$

where S_θ is the backbone’s standard sampler with the trained weights. Test-time editing uses one image-plus-instruction call and one sampling run, without a teacher, masks, decomposed prompts, external planning, or sequentially feeding edited images back into the model.

4 Experiments

We evaluate the paper’s claims in four steps. **Q1. Completion:** Does OCD realize more atomic requests in one editor call?

Q2. Object assignment: Are those changes assigned to the intended objects without merely changing more of the image?

Q3. Training setup: Which target and optimization choices are needed for the gain?

Q4. Efficiency: How close is OCD to structured editors, and how much faster is it at test time?

Experimental Setup

Benchmarks and metrics. ComplexBench-Edit (Wang et al. 2025) contains three atomic requirements per example, including object, attribute, and global edits, with some requests depending on earlier ones. Following its protocol, GPT-5.4 receives the source image, edited image, complete instruction, and current atomic requirement, and returns a score from 0 to 5. We average all 2,289 judgments for the overall score and also report a mean for each edit type. To test object assignment, we use the 164-case multi-object split of OIR/LoMOE (Yang et al. 2024; Chakrabarty et al. 2024). CLIP-T (Radford et al. 2021) measures full-image agreement with the complete instruction; Obj-CLIP measures whether each target region has the requested content; and Bind tests whether each target region matches its intended phrase more strongly than competing phrases from the same instruction. For image-plus-instruction editors, evaluation masks are used only by the metrics; the structured methods described below also receive ground-truth masks at test time.

Baselines and test-time inputs. The main comparison includes editors that receive only the source image and complete instruction, including TurboEdit and InfEdit (Deutch et al. 2024; Xu et al. 2024). We separately report LayerEdit (Fu et al. 2026)—the structured teacher used to construct our data—and LoMOE (Chakrabarty et al. 2024) under their official settings with ground-truth masks and one prompt per atomic request. These two methods use extra inputs at test time, so they are reference methods rather than same-input baselines. They show the quality available when masks and separated prompts remain part of editing.

Training setup and editing time. Teacher outputs are constructed from COCO2017-train images with instance masks (Lin et al. 2014); evaluation images and instructions are excluded. OCD-SDXL uses rank-16 LoRA at 1024^2 , 3,000 steps, and batch size 64 per GPU on eight GPUs (global batch 512; gradient accumulation 1). The FLUX variant uses the same targets and converts its native prediction to a clean latent with the FLUX scheduler. End-to-end editing time is measured on one NVIDIA A800 over six randomly sampled cases. Cache counts and full training and test-time settings are in the supplementary material.

Multi-Edit Completion

Table 2 first compares each OCD model with its own backbone under the same one-shot interface. OCD-SDXL improves the overall score from 1.856 to 2.023, with gains on eight of ten edit types; deletion and text remain weaker. OCD-FLUX improves from 3.473 to 3.567, gaining on addition, change, pose, material, content, and style, while deletion, color, text, and background change slightly in the opposite direction. The gain is therefore broad but not uniform.

The structured methods obtain 2.045 for LayerEdit and 2.060 for LoMOE. OCD-SDXL reaches 2.023 without their masks or separated prompts, and OCD-FLUX raises the same image-plus-instruction interface to 3.567. These results show that more requested changes are completed, but the overall score does not show whether each phrase controls the correct object.

Instruction-to-Object Binding

Table 3 tests whether each instruction phrase controls the intended object. Relative to their backbones, OCD-SDXL raises Bind from 0.4209 to 0.4503 and OCD-FLUX from 0.4082 to 0.4326; both also improve Obj-CLIP. Among single-pass methods, OCD-SDXL has the highest Bind, while OCD-FLUX has the highest CLIP-T and Obj-CLIP.

The metrics separate global agreement from correct object assignment. TurboEdit reaches 0.3257 CLIP-T and 0.2544 Obj-CLIP, yet only 0.4056 Bind; OCD-SDXL improves all three values to 0.3352, 0.2629, and 0.4503. LayerEdit and LoMOE remain higher on Bind at 0.4643 and 0.4668, respectively, when given target masks and separated prompts at test time. Their models and editing pipelines also differ, so this comparison cannot isolate the effect of masks alone.

Background Changes and Metric Meaning

A model could improve completion and binding simply by changing more pixels. We therefore check background behavior in two ways. On explicitly requested background edits, OCD-SDXL improves the ComplexBench score from 1.879 to 2.049, while OCD-FLUX remains close to its strong backbone (4.688 vs. 4.707). The supplementary material also compares source and output pixels outside the annotated edit masks. OCD-SDXL changes BG-LPIPS from 0.3059 to 0.3257 and BG-PSNR from 22.74 to 22.17, indicating slightly more change outside these masks than SDXL img2img. We therefore do not claim improved pixel preservation.

These two measurements answer different questions. A newly added or replaced object may occupy pixels that were background in the source; a later collar in a replace-add-recolor chain has no mask in the source image; and valid shadows, reflections, lighting, or occlusion can extend beyond an object mask. Background and style requests intentionally change even larger regions. This difference is visible across methods: FLUX.1 Kontext obtains 4.707 requested-background completion despite BG-LPIPS/PSNR of 0.3859/18.03, whereas the mask-guided LayerEdit preserves pixels outside the masks more closely at 0.1817/27.97 but obtains only 2.019 requested-background completion. BG-LPIPS and BG-PSNR are useful for measuring changes outside the given masks, but they do not measure whether all requested edits were completed.

Training Components

After measuring completion, binding, and background changes, we ask whether simpler teacher targets are sufficient. Table 4 keeps the SDXL backbone and test-time input fixed while changing the training targets. Complete-output distillation improves CLIP-T to 0.3321 but lowers Bind to 0.4031, showing that a strong complete target does not by itself teach which request belongs to which object. An isolated single-edit target raises Bind to 0.4362, so exposing one request is useful; however, combining independently generated isolated targets returns Bind to 0.4205, near the pretrained 0.4209. Independently generated edits therefore do not automatically reproduce the complete editing context.

There is an important difference between how a target is built and what the student sees. To build Δ_i , the teacher turns request i off while keeping the other requests active, but the per-request student example contains only (x, c_i) . We use Δ_i to link request i to a visible change; we do not assume that this branch can recover every interaction with the other requests. The complete-instruction branch (x, c) teaches dependencies and the final joint result. When an edit cannot be determined from (x, c_i) alone, the per-request target remains incomplete; this is a limitation of the method.

Among the tested variants, the full OCD setup performs best on all three metrics, reaching 0.3352 CLIP-T, 0.2629 Obj-CLIP, and 0.4503 Bind. The version with complete and paired targets but without PCGrad reaches 0.4322 Bind, while isolated supervision reaches 0.4362. These results support the full training setup as a whole: per-request targets built

Method	Overall	Object			Attribute					Global	
		Add	Change	Delete	Color	Pose	Material	Content	Text	Background	Style
SDXL img2img	1.856	1.615	1.893	2.004	1.671	1.770	2.043	2.247	0.618	1.879	2.693
OCD-SDXL	2.023	1.923	2.232	1.640	2.271	2.034	2.221	2.606	0.596	2.049	2.813
FLUX.1 Kontext	3.473	3.930	3.127	3.338	4.135	2.902	3.052	3.424	2.640	4.707	3.188
OCD-FLUX	3.567	4.104	3.392	3.263	4.074	3.011	3.146	3.607	2.431	4.688	3.257
InstructPix2Pix	1.524	1.519	1.630	1.011	2.124	1.164	1.543	1.785	0.539	2.141	2.416
MagicBrush	1.730	1.981	1.857	1.730	1.871	1.098	1.621	1.886	0.427	1.364	1.406
Instruct-CLIP	1.463	1.326	1.559	1.304	1.888	0.967	1.612	1.563	0.539	1.697	2.168
TurboEdit	1.616	1.934	1.775	0.741	1.724	1.754	1.836	2.222	0.663	1.475	2.891
InfEdit	1.629	1.677	1.731	1.044	1.788	1.557	2.034	2.215	0.472	1.960	2.673
ParallelEdits	0.856	1.226	0.784	0.264	0.753	0.885	0.983	1.146	0.112	1.131	2.238
LayerEdit (teacher)	2.045	1.881	2.196	1.848	2.229	2.006	2.246	2.568	0.641	2.019	2.834
LoMOE	2.060	1.947	2.207	1.793	2.304	2.008	2.252	2.641	0.625	2.021	2.842

Table 2: ComplexBench-Edit atomic-requirement scores (0–5). Overall averages all 2,289 requirements; per-type counts are provided in the supplementary material. The Background column measures completion of requested background edits, not pixel preservation outside the edit masks.

Method	CLIP-T $\uparrow$	Obj-CLIP $\uparrow$	Bind $\uparrow$
<i>One call, image + instruction only</i>			
SDXL img2img	0.3162	0.2532	0.4209
OCD-SDXL	0.3352	0.2629	0.4503
FLUX.1 Kontext	0.3304	0.2622	0.4082
OCD-FLUX	0.3482	0.2717	0.4326
InstructPix2Pix	0.3129	0.2614	0.4311
MagicBrush	0.3058	0.2503	0.4209
Instruct-CLIP	0.2976	0.2532	0.4235
TurboEdit	0.3257	0.2544	0.4056
InfEdit	0.3324	0.2600	0.4184
ParallelEdits	0.3301	0.2602	0.4005
<i>Extra inputs at test time (GT masks + per-request prompts)</i>			
LayerEdit (teacher)	0.3268	0.2677	0.4643
LoMOE	0.3346	0.2691	0.4668

Table 3: OIR/LoMOE 164-case multi-object split. Upper block: editors that receive only the image and instruction. Lower block: editors that also receive ground-truth masks and per-request prompts at test time; OCD uses these inputs only during training.

with the other requests active, complete-instruction training, and PCGrad together give the best result. They do not show that paired differences alone are better than isolated targets, or that more teacher data alone explains the gain.

Quality and Editing Speed

We next compare quality and editing time with methods that still use masks and separated prompts at test time. On SDXL, OCD is within 0.022 overall points of LayerEdit and 0.037 of LoMOE on ComplexBench-Edit, while its Bind is lower by 0.0140 and 0.0165, respectively. Thus, OCD comes close to but does not fully match their object-assignment scores. Because the methods receive different inputs and use different editing pipelines, this is not a controlled comparison.

Training target	CLIP-T $\uparrow$	Obj-CLIP $\uparrow$	Bind $\uparrow$
None	0.3162	0.2532	0.4209
Isolated edit	0.3302	0.2605	0.4362
Complete output	0.3321	0.2617	0.4031
Combined isolated edits	0.3321	0.2618	0.4205
Complete + paired, no PCGrad	0.3314	0.2601	0.4322
OCD-SDXL	0.3352	0.2629	0.4503

Table 4: SDXL training ablation with the same backbone and test-time input.

Method	Extra test-time inputs	Time (s)
LayerEdit (teacher)	GT masks + prompts	46.54 $\pm$ 0.15
LoMOE	GT masks + prompts	26.52 $\pm$ 0.67
OCD-SDXL	none	3.67 $\pm$ 0.13

Table 5: Test-time inputs and editing time (mean $\pm$ standard deviation over six randomly sampled cases on one A800).

Test-time editing is much faster. OCD-SDXL runs in 3.67 ± 0.13 seconds per image, versus 26.52 ± 0.67 for LoMOE and 46.54 ± 0.15 for LayerEdit (Table 5), making it $7.2\times$ and $12.7\times$ faster. Teacher decomposition and per-request runs are used only to build the training data; at test time, OCD uses one standard sampling run. Because resolution, sampling steps, and implementations also differ, these numbers compare the complete systems and do not isolate the effect of removing structure.

Qualitative Analysis and Limitations

Figure 3 connects the scores to the motivating failures. In the cross-category swap, several baselines collapse both animals into one category, whereas OCD preserves their phrase-object assignments. In the dependent replace-add-recolor chain, the collar must attach to the newly created dog and then

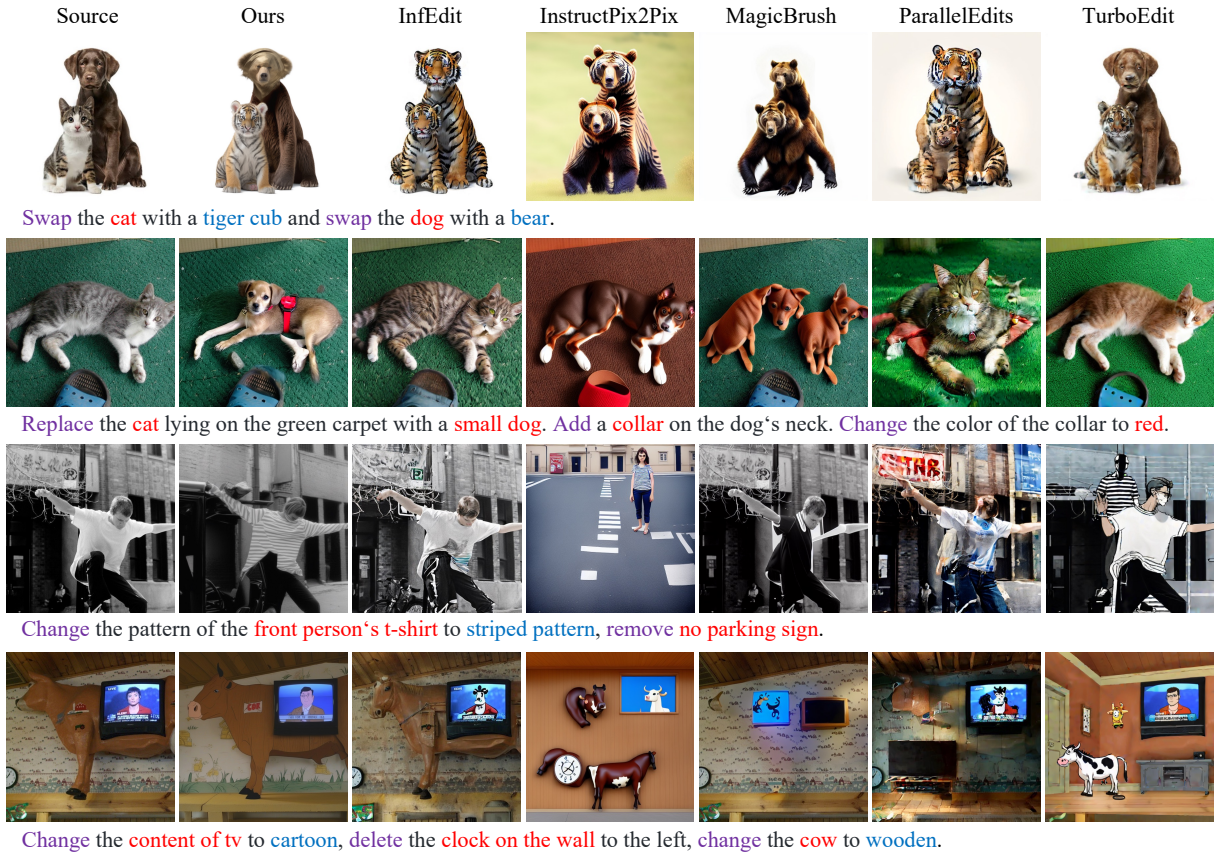


Figure 3: Qualitative comparisons on category swaps, a dependent replace-add-recolor chain, spatial binding, and mixed content/deletion/material edits. In these examples, OCD better avoids the omission, wrong-object assignment, and over-editing exhibited by the baselines.

receive the later color change; MagicBrush duplicates the animal and TurboEdit leaves the source nearly unchanged. The spatial request exposes wrong-instance binding when stripes land on the background person, while the mixed request exposes interference when a local “cartoon” edit spreads across the scene.

The method still has clear limits. Teacher construction requires one leave-one-request-out run per request and therefore grows linearly with instruction length. The student can learn only changes that the teacher produces correctly. Removing bad pairs helps but does not fully solve this problem. Attribute changes can spread beyond enlarged masks, while real shadows, reflections, and lighting can also fall outside those masks. Edits that strongly affect one another and global edits are hard to describe with local differences; complete-instruction training helps but does not remove this limit. ComplexBench scores decrease on deletion and text for both variants, and on color and background for OCD-FLUX. Finally, the training ablation covers one backbone and the evaluation is automatic; a human study remains future work.

5 Conclusion

OCD builds one complete change target and one target for each request from matched teacher outputs, then trains a

standard one-shot editor with both. Across SDXL and FLUX, it improves atomic-request completion and instruction-to-object binding without masks or planning at test time. The ablations support the full training setup—per-request targets, complete-instruction training, and PCGrad—while the speed comparison shows the benefit of using structure only during training and keeping one editor call at test time.

References

- Avrahami, O.; Lischinski, D.; and Fried, O. 2022. Blended diffusion for text-driven editing of natural images. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 18208–18218.
- Brooks, T.; Holynski, A.; and Efros, A. A. 2023. Instruct-pix2pix: Learning to follow image editing instructions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 18392–18402.
- Chakrabarty, G.; Chandrasekar, A.; Hebbalaguppe, R.; and AP, P. 2024. Lomoe: Localized multi-object editing via multi-diffusion. In *Proceedings of the 32nd ACM International Conference on Multimedia*, 3342–3351.
- Chefer, H.; Alaluf, Y.; Vinker, Y.; Wolf, L.; and Cohen-Or, D. 2023. Attend-and-excite: Attention-based semantic guidance for text-to-image diffusion models. *ACM transactions on Graphics (TOG)*, 42(4): 1–10.
- Chen, S. X.; Sra, M.; and Sen, P. 2025. Instruct-clip: Improving instruction-guided image editing with automated data refinement using contrastive learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 28513–28522.
- Couairon, G.; Verbeek, J.; Schwenk, H.; and Cord, M. 2023. Diffedit: Diffusion-based semantic image editing with mask guidance. In *ICLR 2023 (Eleventh International Conference on Learning Representations)*.
- Deutch, G.; Gal, R.; Garibi, D.; Patashnik, O.; and Cohen-Or, D. 2024. Turboedit: Text-based image editing using few-step diffusion models. In *SIGGRAPH Asia 2024 Conference Papers*, 1–12.
- Fu, F.; Huang, M.; Zhang, L.; and Mao, Z. 2026. Layeredit: Disentangled multi-object editing via conflict-aware multi-layer learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 4003–4011.
- Guo, Q.; and Lin, T. 2024. Focus on your instruction: Fine-grained and multi-instruction image editing by attention modulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 6986–6996.
- Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Huang, M.; Cai, J.; Jia, S.; Lokhande, V. S.; and Lyu, S. 2024a. Paralleledits: Efficient multi-aspect text-driven image editing with attention grouping. *Advances in Neural Information Processing Systems*, 37: 22569–22595.
- Huang, Y.; Xie, L.; Wang, X.; Yuan, Z.; Cun, X.; Ge, Y.; Zhou, J.; Dong, C.; Huang, R.; Zhang, R.; et al. 2024b. Smartedit: Exploring complex instruction-based image editing with multimodal large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8362–8371.
- Kim, D.; Lee, J.; and Park, J. 2025. Improving Editability in Image Generation with Layer-wise Memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 7889–7898.
- Labs, B. F.; Batifol, S.; Blattmann, A.; Boesel, F.; Consul, S.; Diagne, C.; Dockhorn, T.; English, J.; English, Z.; Esser, P.; et al. 2025. FLUX. 1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space. *arXiv preprint arXiv:2506.15742*.
- Li, M.; Gu, X.; Chen, F.; Xing, X.; Wen, L.; Chen, C.; and Zhu, S. 2025a. Superedit: Rectifying and facilitating supervision for instruction-based image editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 19206–19215.
- Li, Y.; Chan, K.; Sun, Y.; Lam, C.; Tong, T.; Yu, Z.; Fu, K.; Liu, X.; and Tan, T. 2025b. Moedit: On learning quantity perception for multi-object image editing. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2683–2693.
- Lin, T.-Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; and Zitnick, C. L. 2014. Microsoft coco: Common objects in context. In *European conference on computer vision*, 740–755. Springer.
- Liu, H.; Zou, B.; Zhang, S.; Chen, K.; Liu, R.; and Li, H. 2025. Disentangling Instruction Influence in Diffusion Transformers for Parallel Multi-Instruction-Guided Image Editing. *arXiv preprint arXiv:2504.04784*.
- Pathiraja, B.; Patel, M.; Singh, S.; Yang, Y.; and Baral, C. 2025. RefEdit: A Benchmark and Method for Improving Instruction-based Image Editing Model on Referring Expressions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 15646–15656.
- Podell, D.; English, Z.; Lacey, K.; Blattmann, A.; Dockhorn, T.; Müller, J.; Penna, J.; and Rombach, R. 2024. Sdxl: Improving latent diffusion models for high-resolution image synthesis. In *International Conference on Learning Representations*, volume 2024, 1862–1874.
- Qu, T.; Ke, L.; Zhan, X.; Tang, L.; Liu, Y.; Peng, B.; Yu, B.; Yu, D.; and Jia, J. 2025. RePlan: Reasoning-guided Region Planning for Complex Instruction-based Image Editing. *arXiv preprint arXiv:2512.16864*.
- Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, 8748–8763. PmlR.
- Rassin, R.; Hirsch, E.; Glickman, D.; Ravfogel, S.; Goldberg, Y.; and Chechik, G. 2023. Linguistic binding in diffusion models: Enhancing attribute correspondence through attention map alignment. *Advances in Neural Information Processing Systems*, 36: 3536–3559.
- Schouten, M.; Kaya, M. O.; Belongie, S.; and Papadopoulos, D. P. 2025. POEM: Precise Object-level Editing via MLLM control. *arXiv preprint arXiv:2504.08111*.
- Sheynin, S.; Polyak, A.; Singer, U.; Kirstain, Y.; Zohar, A.; Ashual, O.; Parikh, D.; and Taigman, Y. 2024. Emu edit: Precise image editing via recognition and generation tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8871–8879.

Vapnik, V.; and Vashist, A. 2009. A new learning paradigm: Learning using privileged information. *Neural networks*, 22(5-6): 544–557.

Wang, C.; Zhou, Y.; Wang, Q.; Wang, Z.; and Zhang, K. 2025. Complexbench-edit: Benchmarking complex instruction-driven image editing via compositional dependencies. In *Proceedings of the 33rd ACM International Conference on Multimedia*, 13391–13397.

Wang, Z.; Sha, Z.; Ding, Z.; Wang, Y.; and Tu, Z. 2024. Tokencompose: Text-to-image diffusion with token-level supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8553–8564.

Wei, C.; Xiong, Z.; Ren, W.; Du, X.; Zhang, G.; and Chen, W. 2025. Omniedit: Building image editing generalist models through specialist supervision. In *International Conference on Learning Representations*, volume 2025, 259–271.

Xu, S.; Huang, Y.; Pan, J.; Ma, Z.; and Chai, J. 2024. Inversion-free image editing with language-guided diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 9452–9461.

Yang, Z.; Ding, G.; Wang, W.; Chen, H.; Zhuang, B.; and Shen, C. 2024. Object-aware inversion and reassembly for image editing. In *International Conference on Learning Representations*, volume 2024, 50839–50862.

Yeh, C.-H.; Wang, Y.; Zhao, N.; Zhang, R.; Li, Y.; Ma, Y.; and Singh, K. K. 2026. Beyond simple edits: X-planner for complex instruction-based image editing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 11991–11999.

Yu, Q.; Chow, W.; Yue, Z.; Pan, K.; Wu, Y.; Wan, X.; Li, J.; Tang, S.; Zhang, H.; and Zhuang, Y. 2025. Anyedit: Mastering unified high-quality image editing for any idea. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 26125–26135.

Yu, T.; Kumar, S.; Gupta, A.; Levine, S.; Hausman, K.; and Finn, C. 2020. Gradient surgery for multi-task learning. *Advances in neural information processing systems*, 33: 5824–5836.

Zhang, K.; Mo, L.; Chen, W.; Sun, H.; and Su, Y. 2023. Magicbrush: A manually annotated dataset for instruction-guided image editing. *Advances in Neural Information Processing Systems*, 36: 31428–31449.

Zhao, H.; Ma, X.; Chen, L.; Si, S.; Wu, R.; An, K.; Yu, P.; Zhang, M.; Li, Q.; and Chang, B. 2024. Ultraedit: Instruction-based fine-grained image editing at scale. *Advances in Neural Information Processing Systems*, 37: 3058–3093.

Zhu, H.; Liu, H.; Fu, B.; and Wang, Y. 2025. MDE-Edit: Masked Dual-Editing for Multi-Object Image Editing via Diffusion Models. *arXiv preprint arXiv:2505.05101*.